

Hybrid Transaction Envelopes for Secure Daily Petty-Cash Transactions with Reserved-Value Offline Settlement and Duress Authorization

Author Names Withheld for Review

Affiliation

e-mail address

Abstract—This paper proposes a transaction-level hybrid cryptographic envelope for daily petty-cash and mobile e-payment systems, designed around offline-first operation. For every payment, the client creates a fresh ephemeral P-256 elliptic-curve Diffie-Hellman (ECDH) key pair and combines it with an authenticated long-term bank ECDH public key. HKDF-SHA256 derives a transaction-specific AES-256 key, while AES-256-GCM protects the confidential payment payload and selected associated metadata. A device-bound ECDSA P-256 signing key, released only after operating-system biometric authorization, signs the canonical protected envelope and provides sender-specific authorization. The same authorization step is extended, through key separation and at no added cryptographic cost, to a restricted duress authorization path. Timestamp, nonce, and a globally unique transaction identifier support freshness, while an atomic server-side check-and-commit rule prevents duplicate and concurrent settlement. Because daily petty-cash usage is subject to intermittent connectivity, the same envelope also supports face-to-face offline payment: value reserved online is held by a secure-element (SE) wallet, and a payment is final when the payee’s SE atomically credits it, so later connectivity performs reconciliation, not approval. The principal trade-off is that the online receiver uses a long-term ECDH key to permit asynchronous envelope construction; online envelopes have window-limited forward secrecy through epoch key rotation, whereas offline envelopes use ephemeral keys on both sides.

Index Terms—e-payment, hybrid cryptography, ECDH, HKDF, AES-256-GCM, ECDSA, biometric authorization, replay prevention, idempotency, offline payment, immediate finality, secure element, reconciliation, duress authorization

I. INTRODUCTION

Banking and petty-cash transactions contain sensitive values such as sender, receiver, amount, time, and transaction identifiers. Encryption alone is insufficient: a payment service must also detect modification, authenticate the authorizing user, reject replayed or duplicated requests, protect device keys, tolerate temporary connectivity loss, and preserve correct account state under concurrent processing [1]–[3]. Studies of deployed payment ecosystems further show that serious weaknesses arise from independently added features and from delegated trust between parties [4], [5]. Mobile-payment research increasingly combines transaction-time authorization, platform-backed key protection, and protocol-level replay controls [2], [6]–[8], and recent surveys of payment authentication and NFC payment security emphasize cryptographic and biometric factors as mitigations [9], [10].

This work integrates those requirements into one signed transaction envelope, with intermittent connectivity treated as the normal operating condition for daily petty-cash use rather than an exception [3], [11]–[13]. The design makes four main contributions: (1) an ephemeral-sender/static-receiver ECDH key-establishment path with explicit HKDF context binding [2]; (2) AES-256-GCM authenticated encryption for the payment payload [2], [6]; (3) biometric-gated, device-bound ECDSA authorization over the exact protected envelope, extended via key separation to a restricted duress path with no additional cryptographic operation [6], [7], [9], [14]; and (4) a reserved-value offline mode that reuses the same envelope for face-to-face payment with immediate finality—bank-reserved value in a secure-element wallet, an exactly-once transfer state machine, and reconciliation that never re-approves an accepted payment [3], [11]–[13], [15]. The contribution is protocol and system integration rather than a new cryptographic primitive.

II. SYSTEM AND THREAT MODEL

The protocol contains a client device, an enrolled user, and a receiver service. The receiver owns a long-term P-256 ECDH key pair (SK_R^{dh}, PK_R^{dh}) identified by *KeyID*. The public key is authenticated through the application trust path, while the private key is intended to reside in an HSM or equivalent isolated service. Each enrolled device owns a distinct P-256 ECDSA signing pair (SK_U^{sig}, PK_U^{sig}) ; Section III-A extends this to a second, policy-restricted pair. Signing private keys are non-exportable where supported and require per-operation user authentication through the platform keystore and biometric prompt [6], [16]. For offline operation (Section IV-A), each device additionally contains a secure-element wallet whose trust assumption is stated there.

The network adversary may eavesdrop, record, modify, drop, reorder, duplicate, or replay application messages, capabilities that the contactless-payment literature shows are exercised in practice through eavesdropping, relay, and replay of recorded transaction data [1]. The model also considers sender impersonation, bank-key substitution attempts, duplicate/concurrent request delivery, and opportunistic, time-constrained coercion of the user. A fully compromised server/HSM, a fully malware-controlled device able to invoke authorized operations after authentication, denial-of-service, and coercion outcomes beyond application-level restrictions

remain outside the cryptographic guarantees. TLS remains mandatory; the transaction envelope is an application-layer protection mechanism, not a replacement for transport security [17].

III. CRYPTOGRAPHIC TRANSACTION ARCHITECTURE

ECDH keys are used only for key establishment and ECDSA keys only for signatures, following key-separation practice [18], [19]. A payment message is

$$M = \{S, R, A, T, N, TxID\}, \quad (1)$$

where S and R are sender and receiver identifiers, A is the amount, T is the timestamp, N is a fresh nonce, and $TxID$ is a globally unique transaction identifier.

For each transaction, the sender generates a fresh ephemeral P-256 ECDH pair (esk, ePK) and computes

$$Z = \text{ECDH}(esk, PK_R^{dh}). \quad (2)$$

The ECDH result is not used directly as an AES key. Instead,

$$salt = H(\text{HTE-v1-salt} \parallel S \parallel TxID \parallel N), \quad (3)$$

$$PRK = \text{HKDF-Extract}(salt, Z), \quad (4)$$

and

$$info = \text{HTE-v1/AES-256-GCM} \parallel T \parallel ePK \parallel KeyID. \quad (5)$$

The transaction-specific AES-256 key is

$$K_T = \text{HKDF-Expand}(PRK, info, 32). \quad (6)$$

The 32-byte output K_T is the transaction-specific AES-256 key [20].

The sender generates a fresh 96-bit GCM IV and defines

$$AAD = \{v, S, T, N, TxID, KeyID, ePK\}, \quad (7)$$

where v is the protocol version. The confidential payment content is protected as

$$(C, Tag) = \text{AES-256-GCM}_{K_T}(IV, M, AAD), \quad (8)$$

so successful tag verification authenticates both the ciphertext-protected content and the AAD [21].

After the user confirms the displayed transaction details, biometric authorization releases one ECDSA signing operation:

$$Sig = \text{ECDSA-SHA256}_{SK_U^{sig,k}}(v \parallel KeyID \parallel ePK \parallel IV \parallel C \parallel Tag \parallel AAD), \quad (9)$$

with $k \in \{\text{normal}, \text{duress}\}$ selected by the platform authenticator, as detailed in Section III-A. The transmitted package is

$$P = \{v, KeyID, AAD, ePK, IV, C, Tag, Sig\}. \quad (10)$$

On receipt of P , the receiver performs the following ordered checks: (1) validate protocol version and $KeyID$; (2) verify the ECDSA signature against the registered device public-key set $\{PK_U^{normal}, PK_U^{duress}\}$ and record which key matched;

(3) verify timestamp policy, nonce syntax/state, and $TxID$ format; (4) reject or idempotently return a previously committed $TxID$; (5) compute the corresponding ECDH shared secret and derive K_T using the identical HKDF encoding; (6) verify the GCM tag and decrypt M ; (7) validate amount, account state, limits, receiver, and authorization policy—applying the restricted duress profile of Section III-A if step (2) matched PK_U^{duress} ; and (8) atomically commit the financial update together with the unique $TxID$ record. No balance-changing side effect occurs before all cryptographic and business-rule checks succeed. Figs. 1 and 2 summarize the sender and receiver steps and the online sequence; Section IV-A instantiates the same envelope for offline settlement.

A. Duress Authorization via Key Separation

Routine payment authorization links two facts: the operating system verifies an allowed local authenticator, and successful authorization permits one ECDSA signing operation with the enrolled device key. To support restricted duress operation without adding a cryptographic operation or a network round trip, each device additionally enrolls a second, policy-restricted ECDSA key pair $(SK_U^{sig,duress}, PK_U^{sig,duress})$, gated by a distinct platform authenticator trigger (e.g., a separate enrolled fingerprint or a distinct device credential mapped to a separate keystore alias).

Signing equation (9) is unchanged in form: only the enrolled key selected by the OS authenticator at signing time differs. Because key selection happens entirely inside the platform keystore/secure enclave, no extra ECDH, HKDF, or AES-GCM operation is introduced on the client, and the receiver's added cost is a single extra public-key lookup against a two-entry per-device table at step (2) of the receiver checks in Section III—an $O(1)$ addition.

On a match against $PK_U^{sig,duress}$, the receiver applies a restricted wallet/profile with a preconfigured spend limit L_D ; transactions within this profile are real but bounded, and activation creates a silent risk event for later review and may temporarily restrict sensitive operations. Application-level duress modes with limited transfers, partial-balance display, and silent alerts have been proposed using a separate duress PIN [14]; here the same receiver-side effect is obtained by selecting a distinct signing key inside the platform authenticator, so that the duress signal is carried by the signature itself.

We state the scope of this mechanism explicitly, consistent with Clark and Hengartner's analysis of panic-password schemes [22]: key separation gives a policy-level, non-indistinguishable duress signal. It does not defeat a patient adversary who is aware the mechanism exists and who can compel disclosure of both enrolled credentials, nor does it address lawful compelled-disclosure scenarios. The mechanism is scoped to opportunistic, time-constrained coercion (e.g., street-level robbery), where the adversary cannot verify which credential was used and cannot feasibly obtain both before the transaction must complete. This limitation is inherent to duress-code systems in general.

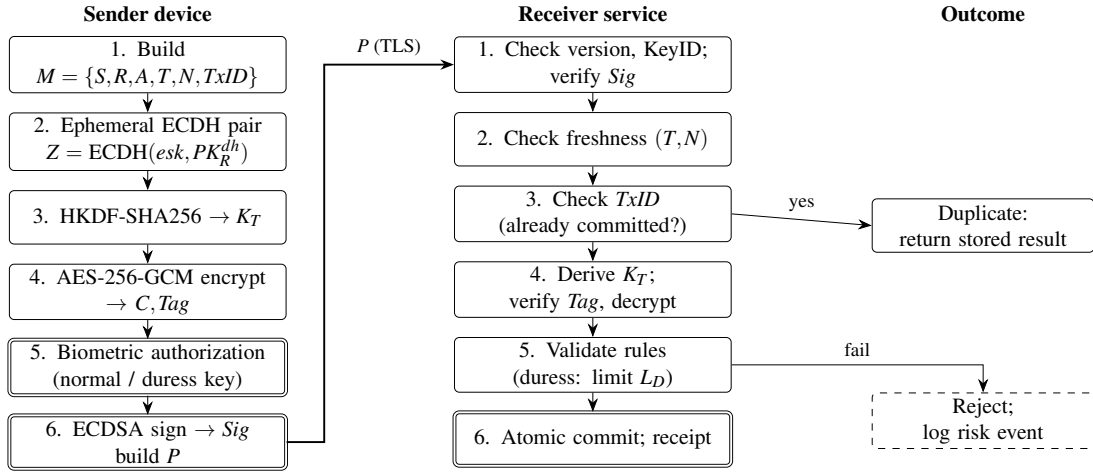


Fig. 1. End-to-end transaction flow. The sender builds M , derives the transaction key, encrypts, and signs; the biometric trigger selects the normal or duress signing key. The receiver applies the ordered checks of Section III: a previously committed $TxID$ returns the stored result, a failed rule check rejects the envelope and logs a risk event (any earlier failed check is rejected in the same way), and only step 6 changes account state.

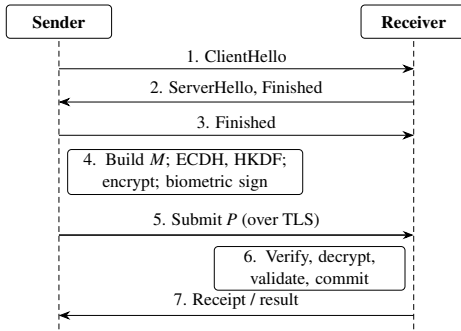


Fig. 2. Online transaction sequence. The envelope is built and signed on the device without any network exchange (step 4) and is then sent as application data over the established TLS 1.3 session (steps 1–3 show the handshake). The receiver verifies the signature, not the biometric itself, and returns a receipt or the stored result for a repeated $TxID$.

IV. OPERATIONAL CAPABILITIES

Figure 2 summarizes the ordinary online path: the sender builds and signs the protected package, submits it immediately, and the receiver performs freshness, signature, decryption, business-rule, and atomic settlement checks. OTP may remain available for account recovery, new-device enrollment, high-risk escalation, or policy exceptions, as OTP-based factors remain widespread in deployed payment authentication [9]; the receiver validates the resulting signature, not the biometric itself.

A. Offline Mode: Reserved-Value Settlement with Immediate Finality

Petty-cash payments often occur without connectivity (markets, transit, rural areas), and a merchant must not hold a claim that may later fail, so offline payments are *settled* between devices. As in pre-funded hardware-wallet designs [11]–[13], [15], [23], [24], the mode runs *online reservation* $\rightarrow$ *offline*

transfer $\rightarrow$ *immediate finality* $\rightarrow$ *later reconciliation* (Fig. 5), reusing the envelope of Section III unchanged.

Reservation and wallet. A device cannot know a live account balance offline, so while online the bank executes in one ledger transaction

$$B_u^{\text{on}} \leftarrow B_u^{\text{on}} - L, \quad \text{Res}_W \leftarrow \text{Res}_W + L, \quad (11)$$

with $B_u^{\text{on}} \geq L$ and $\text{Res}_W + L \leq L_{\text{max}}$: of BDT 10,000, reserving 2,000 leaves 8,000 spendable online and 2,000 only offline. Without reservation the payer could also spend the value online, forcing the bank to reject an accepted payment or pay twice. A secure element (SE) holds all offline value state (balance bal_W , received value rcv_W , counter c_W , in-flight transfers, non-exportable sk_W , the device keys $\{PK_U^{\text{normal}}, PK_U^{\text{duress}}\}$, time bound t_W , revocation list); the application only transports and displays. The bank verifies attestation of PK_W and issues

$$\text{OWC}_W = \text{Sig}_{SK_I}(W \| PK_W \| e_W \| t_{\text{iss}} \| t_{\text{exp}} \| \pi_W), \quad (12)$$

with limits $\pi_W = (a_{\text{max}}, L_{\text{max}}, H_{\text{max}}, K_{\text{inf}})$, never a balance. A load $\text{LC}_p = \text{Sig}_{SK_I}(\text{LOAD} \| W \| p \| L)$ applies only if p is the SE's next load number, so resending is idempotent.

Payment (Fig. 3). (i) The payee's SE picks $r = c_R + 1$, nonce n_R , and an ephemeral pair (esk_R, ePK_R) , reserves holding capacity (H_{max}), and emits

$$\begin{aligned} \text{Req} &= \langle W_R, r, n_R, a, ePK_R, H(\text{OWC}_R) \rangle, \\ M_1 &= \langle \text{Req}, \text{Sig}_{\text{sk}_R}(\text{Req}), \text{OWC}_R \rangle. \end{aligned} \quad (13)$$

(ii) The payer's application builds M of (1) with $S = W_S$, $R = W_R$, $A = a$, $N = n_R$, $TxID = H(W_R \| r \| n_R \| W_S)$ and runs (2)–(10) *unchanged* with $PK_R^{\text{dh}} := ePK_R$, $\text{KeyID} := H(ePK_R)$; biometrics select the normal or duress key (Section III-A), and both ECDH keys are ephemeral (forward secrecy). (iii) The payer's SE verifies OWC_R , the request, expiry against t_S , revocation, and the binding of the AAD of

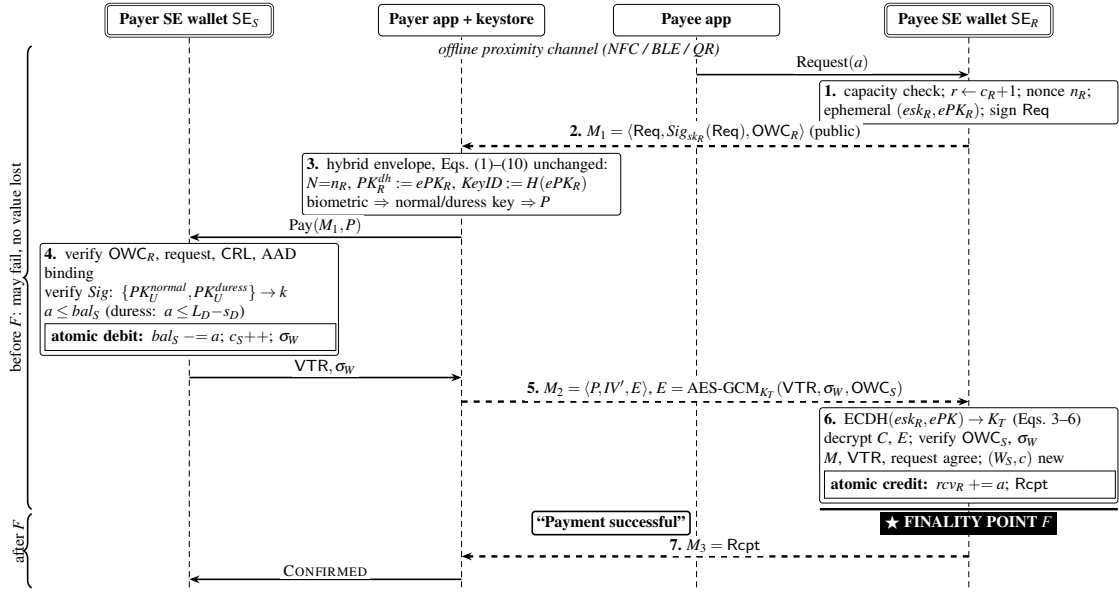


Fig. 3. True offline payment sequence; neither device is online. Step 3 is the unchanged hybrid envelope of Eqs. (1)–(10) with the payee’s ephemeral key in place of the bank key. The payer’s SE debits atomically at step 4 (value then in flight, recoverable exactly once); the payment is final at F , the payee SE’s atomic credit.

P to the request; performs receiver step (2) itself on Sig ; and requires $a \leq bal_S$ (duress: $a \leq L_D - s_D$). It then atomically debits bal_S , increments c_S , marks $TxID$ COMMITTED, and signs

$$\begin{aligned} VTR &= \langle TxID, W_S, e_S, c_S, W_R, r, n_R, a, H(P) \rangle, \\ \sigma_W &= \text{Sig}_{sk_W}(VTR). \end{aligned} \quad (14)$$

(iv) The application sends $M_2 = \langle P, IV', E \rangle$, reusing K_T of (6) under a fresh IV:

$$E = \text{AES-256-GCM}_{K_T}(IV', VTR \parallel \sigma_W \parallel \text{OWC}_S, TxID). \quad (15)$$

(v) The payee’s SE derives K_T via (2)–(6), decrypts C and E , verifies OWC_S and σ_W , checks that M , VTR , and the request agree and (W_S, c_S) is new, then atomically credits $rcv_R += a$, closes r , and returns $\text{Rcpt} = \text{Sig}_{sk_R}(\text{RCPT} \parallel TxID \parallel H(VTR))$.

Finality. The payment is final when this credit commits (point F), after which the payee shows “Payment successful.” Before F it can fail (insufficient balance, cancellation, invalid credential, link loss); after F no retransmission, abort, or reconciliation removes it. Encryption gives only confidentiality; Sig and σ_W authorize, and the SE’s atomic debit and counter prevent double spending.

Atomicity and recovery (Fig. 4). Two-device atomic commit over a lossy link needs a trusted third party [25], so two local atomic commits are joined by an in-flight state and payee-signed evidence. The payer’s entry becomes CONFIRMED on a valid Rcpt , or VOIDED (re-crediting a) on an abort proof $\text{Abt} = \text{Sig}_{sk_R}(\text{ABORT} \parallel W_R \parallel r \parallel n_R)$; the payee’s SE signs at most one of them per request, so value returns only if never credited. A lost message is handled by resending the identical M_2 , which returns the stored Rcpt without a second credit;

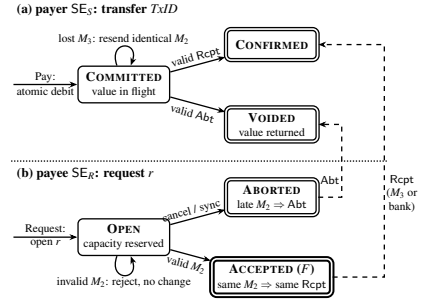


Fig. 4. Per-transfer state machines: (a) payer SE, (b) payee SE. Double borders mark terminal states; dashed arrows are payee-signed evidence. No transition leaves ACCEPTED.

separated parties are resolved when the bank forwards Rcpt or Abt at synchronization.

Double spending and trust assumption. We assume the SE executes as specified (non-extractable keys, no rollback or cloning, atomic commands) [24]; a wallet then cannot spend more than was loaded, and backups, SQLite edits, reinstallation, or clock changes touch no SE state (expiry uses $\hat{t}_W$, advanced only by bank-signed beacons). A broken SE is detected, not prevented: clones or rollbacks yield two SE-signed records with the same (W, c) , and losses are issuer risk bounded by $a_{\max}$, $H_{\max}$, and credential validity [26].

Received value and duress. Received value is final but spendable offline only after reconciliation, so value from a broken SE cannot spread before detection (traceable transferable cash grows per transfer [27]). Duress key separation is unchanged: the payer’s SE identifies the key from Sig and caps spending at L_D ; the payee verifies only σ_W , so a coercing payee cannot detect duress, and the bank repeats step (2) at

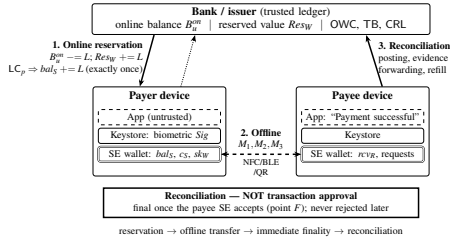


Fig. 5. Offline-mode lifecycle: (1) online reservation and exactly-once load; (2) face-to-face transfer, both devices offline; (3) reconciliation. Reconciliation is not transaction approval.

reconciliation.

Reconciliation and revocation. On reconnection the SE exports its signed hash-chained log; the bank posts a transfer iff σ_W verifies, VTR names the payee, and $TxID$ recomputes (step (8) check-and-commit), never re-checking the payer’s balance, rules, or current revocation, because the value came from Res_S . Signed revocation lists are gossiped between devices, but an offline device cannot learn of a revocation instantly; exposure is bounded by credential validity [28], and payments accepted under stale knowledge are honored. *Example:* Alice (reserved BDT 2,000) pays Bob (reserved 500) BDT 300 offline; at F Alice’s SE holds 1,700 and Bob’s 500+300. The bank later posts it (Bob online 2,500→2,800); Bob cannot lose it even if Alice spends online.

In the online path, the client ECDH key is ephemeral, but the receiver ECDH key is long-term. This supports asynchronous envelope construction because a validated receiver public key can be cached. The cost is that online envelopes lack full forward secrecy against later compromise of SK_R^{dh} : an attacker that recorded an old ePK and envelope could recompute the old transaction key. The offline mode is unaffected because ePK_R is ephemeral.

Epoch key rotation. To bound the impact of a later compromise of the receiver ECDH key, the bank rotates (SK_R^{dh}, PK_R^{dh}) every epoch Δ (e.g., 24 h) and issues each new public key with its *KeyID* and validity interval, signed by the bank. Retired private keys are retained only for the grace period needed to process envelopes still in transit or being retried, and are then securely destroyed in the HSM; envelopes naming a destroyed *KeyID* are rejected. The envelope of Eqs. (1)–(10) is unchanged. Forward secrecy is therefore window-limited: compromise of the current key exposes only envelopes of that epoch and of any retired key still inside its grace period (at most Δ plus the grace period), not earlier traffic. This is weaker than per-transaction forward secrecy, which the offline mode already provides through ephemeral keys on both sides. One-time key pools would give stronger protection but add per-payment key storage, refill, and deletion logic, so we leave them as future work for high-value deployments.

The elliptic-curve envelope also has lower public-key overhead than an illustrative RSA-2048 key-transport plus RSA-PSS construction. With compressed P-256 public-key encoding (33 bytes), a 96-bit GCM IV (12 bytes), a 128-bit tag (16

TABLE I
REPRESENTATIVE PROTOTYPE VALIDATION RESULTS.

Test	Observed result
Valid signed envelope	Settled (200)
1-bit alteration in protected fields	Rejected (403)
Invalid version / <i>KeyID</i>	Rejected (400)
Unenrolled signing key	Rejected (403)
Stale timestamp	Rejected (400)
Reused nonce with new <i>TxID</i>	Rejected (409)
100 identical parallel submissions	1 settled; 99 rejected
Duress payment: 100 / 300, $L_D = 250$	Settled / rejected

bytes), and a raw P-256 ECDSA $r||s$ signature (64 bytes), fixed cryptographic overhead is approximately 125 bytes, excluding AAD, identifiers, framing, and serialization; this is an encoding-level estimate, not an implementation benchmark.

The design has explicit limits. Hardware-backed key protection reduces extraction risk but does not make a fully compromised endpoint trustworthy. Biometric authorization proves local user presence according to platform policy; the receiver’s cryptographic evidence remains the ECDSA signature, which avoids relying on an unverified client-side authentication assertion of the kind whose unconditional acceptance has been exploited in digital-wallet ecosystems [5]. The envelope does not prevent denial of service, traffic analysis, endpoint malware, receiver compromise, or fraudulent transactions that are cryptographically valid but socially engineered. Proximity relay attacks [1], [29] are outside the scope of the remote envelope; offline, each payment needs explicit biometric authorization, so a relay cannot make a phone pay silently. Offline settlement rests on SE integrity. Secure deployment further depends on correct randomness, canonical encoding, TLS, certificate/key validation, HSM/Keystore configuration, database transactions, monitoring, and key-revocation procedures [17].

V. IMPLEMENTATION AND EVALUATION

A reference prototype was implemented to validate the protocol of Sections III–IV. It consists of an Expo/React Native (TypeScript) client that builds and signs the envelope and a Flask/Python receiver using the `cryptography` library and PostgreSQL. A Kotlin Android Keystore module provides device-bound signing keys, with StrongBox requested where supported and TEE-backed storage as fallback [16]. The receiver holds a long-term P-256 ECDH key identified by *KeyID*; each transaction uses a fresh ephemeral P-256 pair, HKDF-SHA256, and AES-256-GCM. User authentication is required for key use, while a second policy-restricted key under a separate keystore alias implements the duress path.

The prototype uses P-256/SHA-256, protocol version $v = 1$, a 96-bit CSPRNG GCM IV, a 300-s timestamp acceptance window, a 24-hour key-rotation grace period, and a 24-hour nonce-validity period. The duress spending limit is $L_D = 250$.

Functional validation included a cross-language client/receiver test and a PostgreSQL-backed concurrent-submission test. Tampering, freshness, key-registration, and

TABLE II
MEASURED CRYPTOGRAPHIC COST ON THE PROTOTYPE WORKSTATION.

Operation	Mean (ms)	p95 (ms)
Client ephemeral ECDH key generation	0.387	0.467
Client ECDH shared-secret computation	2.170	2.296
HKDF-SHA256	0.040	0.058
AES-256-GCM encryption	0.060	0.090
ECDSA signing (software)	0.486	0.601
Envelope construction	3.142	3.327
Receiver ECDSA verification	0.081	0.086
Receiver key derivation	0.082	0.086
GCM verification and decryption	0.006	0.007
Receiver verification	0.169	0.178

duress-limit cases produced the expected receiver decisions. For 100 concurrent submissions of the same $TxID$, database transaction isolation and row locking resulted in exactly one settlement, confirming the atomic check-and-commit property. The duress path required no additional client cryptographic operation because the same envelope bytes are signed using the policy-restricted key; the receiver only performs the corresponding additional public-key verification.

The measurements used 500 client iterations and 300 receiver iterations on an Intel Core i5-14400 Linux x86-64 workstation, with Node.js v22.22.1 for client cryptographic primitives and Python 3.14.4 for receiver primitives. They therefore characterize the prototype implementation rather than smartphone or deployed-server performance. Envelope construction required approximately 3.1 ms on average (3.33 ms at p95), dominated by ECDH computation, while the receiver's cryptographic verification path required approximately 0.17 ms (0.18 ms at p95), excluding network transfer, database commit, and business-rule processing. The software signing measurement does not represent Android Keystore hardware-backed signing, which was not timed.

A sample prototype envelope serialized as JSON with hexadecimal binary fields occupied 779 bytes; its fixed cryptographic overhead was 125 bytes before AAD, identifiers, framing, and serialization.

Offline mode. A keystore protects keys but offers no atomic, rollback-protected balance, so the React Native/Keystore prototype cannot hold offline value; the wallet needs an issuer-controlled SE applet, reached on Android via the Open Mobile API [30] and installed with device-maker or operator cooperation (iOS SE access is vendor-controlled) [24]. The offline protocol was validated with an executable Python reference model of Eqs. (1)–(15), the SE, and the bank; Table V is generated from its 20 checks, including the epoch-rotation check of Section IV. M_1 , M_2 (incl. the 423-byte P), and M_3 take 615, 1,111, and 190 bytes. SE timings and two-phone latency are future work.

VI. CONCLUSION

The proposed framework combines fresh per-transaction ECDH/HKDF key establishment, AES-256-GCM authenticated encryption, and biometric-gated device ECDSA authorization in one protected transaction envelope, extended

to a restricted duress path through key separation at no added cryptographic cost. Timestamp, nonce, unique transaction identifiers, and atomic receiver idempotency address replay and duplicate settlement. Because daily petty-cash use is characteristically subject to intermittent connectivity, the same envelope also supports face-to-face offline payment with immediate finality: value reserved online is held by a secure-element wallet, the payer's SE authorizes the unchanged envelope and debits atomically, and the payment is final at the payee SE's atomic credit, without introducing independent ad-hoc cryptographic paths of the kind that have weakened layered payment specifications [4]. Reconciliation posts, forwards evidence, and replenishes; it does not decide whether an honest merchant is paid. The prototype evaluation demonstrates the functional realization of the online mechanisms and reports low cryptographic processing costs on the test workstation, and the reference model validates the offline protocol logic. The design is intentionally explicit about window-limited online forward secrecy, about the non-indistinguishable nature of the duress mechanism, and about the fact that offline finality rests on secure-element integrity.

REFERENCES

- [1] M. Mehr Nezhad, F. Hao, G. Epiphaniou, C. Maple, and T. Yunusov, "SoK: Security of EMV contactless payment systems," arXiv:2504.12812, 2025.
- [2] A. Salim, A. M. Sagheer, and L. Yaseen, "Design and implementation of a secure mobile banking system based on elliptic curve integrated encryption schema," in *Applied Computing to Support Industry: Innovation and Technology*, M. Khalaf, D. Al-Jumeily, and A. Lisitsa, Eds. Cham, Switzerland: Springer, 2020, pp. 424–438.
- [3] B. Poudel, S. Carey, R. Flynn, C. Narayan, R. Payne, E. Satrasala, S. Spooner, and J. Lovejoy, "A robust risk framework for offline payments," Board of Governors of the Federal Reserve System, Washington, DC, USA, Finance and Economics Discussion Series 2025-105, Dec. 2025.
- [4] G. Pavlides, A. Clee, I. Boureanu, and T. Chothia, "More is less: Extra features in contactless payments break security," in *Proc. 34th USENIX Security Symp.*, 2025.
- [5] R. H. Anwar, S. R. Hussain, and M. T. Raza, "In wallet we trust: Bypassing the digital wallets payment security for free shopping," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024.
- [6] M. M. A. M. Alsaedy, A. Z. Ghalwash, A. A. E. Yousif, and S. M. Azzam, "E-AAPIV: Merkle tree-based real-time Android manifest integrity verification for mobile payment security," *J. Cyber Secur.*, vol. 7, no. 1, pp. 653–674, 2025.
- [7] J. Sturgess and I. Martinovic, "A mobile payment scheme using biometric identification with mutual authentication," in *Proc. 22nd Int. Conf. Security and Cryptography (SECRYPT)*. SciTePress, 2025, pp. 577–585, arXiv:2409.17181.
- [8] C. Wu and Q. Zeng, "Do you see how I pose? Using poses as an implicit authentication factor for QR code payment," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024.
- [9] P. T. Tran-Truong, M. Q. Pham, H. X. Son, D. L. T. Nguyen *et al.*, "A systematic review of multi-factor authentication in digital payment systems: NIST standards alignment and industry implementation analysis," *J. Syst. Archit.*, vol. 162, p. Art. no. 103402, May 2025.
- [10] P. Onumadu and H. Abroshan, "Near-field communication (NFC) cyber threats and mitigation solutions in payment transactions: A review," *Sensors*, vol. 24, no. 23, p. Art. no. 7423, Nov. 2024.
- [11] J. Yoon and Y. Kim, "Offline payment of central bank digital currency based on a trusted platform module," *J. Cybersecur. Privacy*, vol. 5, no. 2, p. Art. no. 14, Apr. 2025.
- [12] A. V. S. K. Reddy and G. Banda, "ElasticPay: Instant peer-to-peer offline extended digital payment system," *Sensors*, vol. 24, no. 24, p. Art. no. 8034, Dec. 2024.

TABLE III
THREAT-TO-CONTROL MAPPING FOR THE PROPOSED FRAMEWORK.

Threat / Failure Mode	Primary Mechanism(s)	How the Framework Responds
Payload disclosure / passive eavesdropping Ciphertext, IV, AAD, or metadata tampering Masquerade / sender impersonation	Per-transaction ECDH + HKDF-SHA256 + AES-256-GCM AES-GCM authentication tag + ECDSA signature Registered device ECDSA public key + non-exportable private key + biometric-gated key use	Sender and receiver derive the same fresh K_T without transmitting it; an observer sees only public envelope metadata and ciphertext. GCM rejects unauthorized modification of protected plaintext/AAD; ECDSA additionally binds ePK , $KeyID$, IV, ciphertext, tag, and freshness metadata. A party without the enrolled private signing key cannot produce a valid authorization signature; the receiver verifies against the registered device public-key set.
Replay of an old valid transaction (cf. pre-play in EMV [1])	Timestamp T + nonce N + unique $TxID$ + freshness policy	Stale requests are rejected by policy; previously committed identifiers are handled idempotently instead of being settled again.
Concurrent replay / duplicate settlement	Database uniqueness on $TxID$ + atomic check-and-commit	Simultaneously arriving valid duplicates cannot race into multiple balance updates.
Receiver-key substitution / active MITM against ECDH Stolen app files / private-key extraction attempt	Authenticated receiver ECDH public key + $KeyID$ + TLS/certificate validation Platform Keystore/StrongBox + non-exportable ECDSA key + per-operation authentication	The sender accepts only a receiver key delivered through the authenticated trust path, bound into HKDF context, AAD, and the signed envelope. Copying application storage should not reveal the private signing key when hardware-backed storage is available.
Connectivity loss, retry storms, or duplicate resubmission	Immutable signed envelope + same $TxID$ + receiver idempotency + backoff	The sender resends one signed identity; retries do not mint new identifiers, so repeated synchronization converges on one receiver result.
Offline double spending, cloning, rollback	Reserved value in SE + atomic debit + counter	Prevented under the SE assumption; a broken SE is detected at reconciliation, losses are issuer risk.
Offline MITM, lost/duplicated messages, revoked credential	VTR binding + σ_W + Rcpt/Abt + short-lived OWC	Altered data rejected before debit/credit; one credit per transfer; revocation bounded by credential validity.
Forced-access / coercive-use scenario [14], [22]	Restricted duress key/profile + preconfigured limit + silent risk event	The session exposes only restricted account state and bounded real transfers (offline: SE-enforced L_D); scope is limited to opportunistic, time-constrained coercion.

TABLE IV
COMPARISON WITH RELATED MOBILE-/CARD-PAYMENT SECURITY MECHANISMS.

Dimension	EMV (chip/contactless) [1], [4]	PCI Tokenization	Signal X3DH	This work (HTE)
Key establishment	Static symmetric/asymmetric card keys, issuer-derived	None – token vault replaces PAN	Ephemeral-ephemeral + one-time prekeys (X25519)	Ephemeral sender + static receiver ECDH, P-256 (offline: ephemeral payee key)
Payload confidentiality	Card/issuer MAC or encryption (scheme-dependent)	Token carries no payment data; PAN encrypted at vault	Double Ratchet (AES + HMAC chains)	AES-256-GCM with HKDF-derived per-transaction key
Sender authorization	PIN / signature / CDA cryptogram	Merchant/API credential, not sender-biometric	Identity-key signature at session start only	Biometric-gated ECDSA over the exact envelope, every transaction
Replay / duplicate protection	Application transaction counter	Token single-use or vault-side dedup	Message counters / ratchet state	Timestamp + nonce + unique $TxID$ + atomic check-and-commit; offline: SE counter
Offline capability	Offline data authentication, limited	N/A (online vault lookup)	Not designed for offline payment	Reserved-value SE wallet; final at payee SE; reconciliation, not approval
Duress / coercion handling	Not specified	Not specified	Not specified	Restricted-profile duress key with spend limit + silent risk event (scope-limited)
Forward secrecy	N/A	N/A	Strong (per-message ratchet)	Online: window-limited (epoch rotation with key destruction); offline: ephemeral-ephemeral
Formal verification	Long-standing, field-tested	Industry-audited (PCI DSS)	Formally analyzed in literature	Not yet performed (future work)

- [13] H. Si, Y. Huang, G. Li, Y. Qi, W. Chen, and Z. Gao, “A method for continuous dual-offline payment of cryptocurrency based on asset credentials,” *Sensors*, vol. 26, no. 10, p. Art. no. 3039, May 2026.
- [14] I. O. Ibraheem and M. T. Jidda, “A duress-enabled mobile banking system for coercion-resistant transactions,” Research Square, preprint (not peer reviewed), Dec. 2025.
- [15] BIS Innovation Hub Nordic Centre, “Project Polaris: Handbook for offline payments with CBDC,” Bank for International Settlements, Innovation Hub project report, May 2023.
- [16] R. Mayrhofer, J. Vander Stoep, C. Brubaker, D. Hackborn, B. Bonné, G. S. Tuncay, R. Piqueras Jover, and M. A. Specter, “The Android platform security model (2023),” arXiv:1904.05572, updated Dec. 2023, 2023.
- [17] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” IETF RFC 9846, Jul. 2026.
- [18] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, “Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography,” NIST, Special Publication 800-56A Rev. 3, Apr. 2018.
- [19] National Institute of Standards and Technology, “Digital signature standard (DSS),” NIST, FIPS PUB 186-5, Feb. 2023.
- [20] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” IETF RFC 5869, May 2010.
- [21] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” NIST, Special Publication 800-38D, Nov. 2007.
- [22] J. Clark and U. Hengartner, “Panic passwords: Authenticating under duress,” in *Proc. 3rd USENIX Workshop on Hot Topics in Security (HotSec '08)*, San Jose, CA, USA, Jul. 2008.
- [23] European Central Bank, “Progress on the preparation phase of a digital euro – First progress report,” European Central Bank, Frankfurt am Main, Germany, Tech. Rep., Jun. 2024.
- [24] E. Androutaki, A. De Caro, K. El Khiyaoui, R. Gay, R. Mercer, and A. Sorniotti, “Secure and privacy-preserving CBDC offline payments using a secure element,” Cryptology ePrint Archive, Paper 2024/1746, 2024.

TABLE V
OFFLINE-MODE AND KEY-ROTATION VALIDATION (GENERATED BY THE
REFERENCE MODEL).

Test	Expected behavior	Result
Offline payment A→B, 300; reconciliation	Final at F ; posted once	Pass
Low balance, MITM, replay, tamper, no biometric	Refused; one credit	Pass
Lost M_2/M_3 , retransmit, separation	Exactly one outcome	Pass
Backup restore; cloned SE (outside model)	No effect; clone detected	Pass
Revoked / expired credential	Refused or bounded	Pass
Concurrent spends; holding cap; loads	One commit; exactly-once	Pass
Duress key, $L_D=250$	Cap holds; silent event	Pass
Online epoch key rotation (Sec. IV)	Destroyed after grace; unreadable	Pass
Fault injection: 300 runs, 2930 pays, 1494 drops	Value conserved	Pass

- [25] H. Pagnia and F. C. Gärtner, “On the impossibility of fair exchange without a trusted third party,” Darmstadt University of Technology, Tech. Rep. TUD-BS-1999-02, 1999.
- [26] C. Beer, S. Zingg, K. Kostianen, K. Wüst, V. Capkun, and S. Capkun, “PayOff: A regulated central bank digital currency with private offline payments,” arXiv:2408.06956, 2024.
- [27] D. Chaum and T. P. Pedersen, “Transferred cash grows in size,” in *Advances in Cryptology – EUROCRYPT ’92*, ser. LNCS, vol. 658. Springer, 1993, pp. 390–407.
- [28] R. L. Rivest, “Can we eliminate certificate revocation lists?” in *Financial Cryptography (FC ’98)*, ser. LNCS, vol. 1465. Springer, 1998, pp. 178–183.
- [29] D. Coppola, G. Camurati, C. Anliker, X. Hofmeier, P. Schaller, D. Basin, and S. Capkun, “PURE: Payments with UWB RELay-protection,” in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024.
- [30] Android Open Source Project, “CTS test for secure element: Open Mobile API and access control,” <https://source.android.com/compatibility/cts/secure-element>, accessed Oct. 2026.